

Pie: A Programmable Serving System for Emerging LLM Applications

1 Paper summary

This paper presents Pie, which is a programmable serving system for LLMs. The main motivation is that newer applications, like reasoning workflows or agent-based systems, are more complicated than normal text generation and don't fit very well with existing serving architectures.

Most current serving systems are built around a fixed token-generation loop. This works fine for simple text completion, but it becomes limiting when applications need custom decoding behavior, more control over KV caches, or integration with external tools and APIs.

Pie tries to solve this by breaking the generation process into smaller components called handlers. These handlers manage things like embedding, sampling, forward passes, or KV-cache operations. Then user-defined programs, called inferlets, orchestrate these handlers through an API. Basically, applications get much more direct control over how generation happens.

The inferlets run inside WebAssembly, which provides some isolation and also allows support for multiple programming languages. This makes the system more flexible, even if it also adds another layer of complexity.

The evaluation covers several advanced techniques, like speculative decoding, constrained decoding, and agentic workflows. The results show that Pie has only a small overhead on standard workloads, while giving noticeable improvements on more advanced workflows, especially in terms of throughput and latency.

2 Contributions

The first contribution is identifying the limitation of monolithic serving systems. The paper argues that current designs are too rigid for newer LLM applications, which is important because many modern workflows require more control than standard APIs provide.

The second contribution is the Pie architecture itself. By exposing fine-grained handlers and inferlets, the system allows applications to customize decoding behavior, KV-cache usage, and interactions with external services. This makes the serving system much more programmable compared to traditional approaches.

The third contribution is showing that this flexibility can still be efficient. Using WebAssembly and a layered runtime, Pie keeps the overhead relatively low while improving performance for more complex workflows. This is important because otherwise programmability would probably not be practical.

3 Problems

One limitation is that the added flexibility also makes the system more complicated to use. Developers now need to think about inferlets, orchestration, and cache management instead of relying on a simpler serving API. This gives more control, but also increases the chance of mistakes or inefficient implementations.

Another issue is that the strongest improvements mostly appear in advanced workflows. For standard text generation, Pie mainly performs similarly to existing systems, sometimes with a small overhead. So the benefits are very workload-dependent.

A third point is that exposing low-level programmability could create problems in shared environments. Different inferlets may compete for resources in unpredictable ways, and fairness or isolation could become harder to manage. WebAssembly helps with sandboxing, but it probably doesn't solve everything completely.